SISTEM DONOR DARAH

Nama	Piere Valkyrie
NIM	2410511152
Kelas	B
Dosen	Muhammad Panji Muslim, S.Pd., M.Kom
Semester	Genap TA. 2025/2026

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA
FAKULTAS ILMU KOMPUTER
S1 INFORMATIKA

2026

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Keperluan terhadap pengurusan bakalan darah dan permintaan dari hospital memerlukan sistem yang pantas, responsif dan boleh dipercayai. Pengurusan secara manual atau dengan sistem monolitik konvensional seringkali menghadapi kendala seperti *bottleneck* prestasi apabila tedapat lonjakan permintaan darah yang mendesak.

Untuk mengatasi cabaran tersebut, pendekatan *Service-Oriented Architecture* (SOA) dengan implementasi *microservices* menjadi penyelesaian yang relevan. Setiap domain perniagaan dipisahkan menjadi servis bebas yang boleh dibangunkan dan di-scale secara berasingan. Laporan ini mendokumentasikan pembangunan Sistem Donor Darah berasaskan *microservices* yang terdiri dari auth-service, donor-service, request-service, dan API Gateway sebagai *single entry point.*

1.2 Studi Kasus

Studi kasus yang dikerjakan adalah Sistem Donor Darah dengan ruang lingkup seperti berikut:

- Register & Login pengguna menggunakan email.
- Melihat stok darah yang tersedia.
- Membuat permintaan darah oleh pihak hospital yang membutuhkan Token JWT.
- Autentikasi dengan JWT dan Google OAuth 2.0 (NIM genap)

1.3 Tujuan

- Membangunkan sistem berasaskan microservices dengan 3 servis bebas dan 1 API Gateway.
- Membangunkan donor-service menggunakan kerangka PHP MVC (Laravel 11).
- Menggunakan Node.js dan Express untuk servis lain (auth-service, request-service, dan Gateway).
- Mengimplementasikan pangkalan data yang berbeza mengikut keperluan servis (MySQL dan MongoDB).
- Mengimplementasikan pengesahan JWT dan Google OAuth 2.0.

BAB 2 ARSITEKTUR SISTEM

2.1 Gambaran Umum Arsitektur

Sistem Donor Darah dibangunkan menggunakan arsitektur microservice yang terdiri dari empat komponen utama:

Service	Teknologi	Database	Port	Fungsi
API Gateway	Node.js + Express	—	8000	Single entry point
auth-service	Node.js + Express	MySQL (auth_db)	3001	Autentikasi (JWT) & Google OAuth
donor-service	Laravel 11 (PHP MVC)	MySQL (donor_db)	3002	Pengurusan stok darah (GET)
request-service	Node.js + Express	MongoDB (blood_request_db)	3003	Menguruskan permintaan darah oleh hospital.

2.2 Justifikasi Pemisahan Service vs Monolitik

Pendekatan microservice dipilih dibandingkan monolitik dengan justifikasi sebagai berikut:

2.2.1 Alasan Pemisahan auth-service

Fitur autentikasi seperti login dibutuhkan oleh hampir semua bagian aplikasi. Dengan memisahkannya menjadi auth-service sendiri, proses pengecekan Token JWT cukup dilakukan di satu tempat, yaitu di API Gateway. Jika menggunakan sistem monolitik, setiap ada perubahan pada sistem login, kita harus mengotak-atik seluruh kode aplikasi.

2.2.2 Alasan Pemisahan ticket-service

Pengelolaan data stok darah membutuhkan struktur pangkalan data yang saling berelasi dengan kuat. Oleh karena itu, service ini dibangun secara khusus menggunakan pola MVC pada Laravel 11. Dengan dipisah, service ini bisa menangani lonjakan pengunjung yang ingin mengecek stok darah tanpa membuat layanan lain ikut menjadi lambat.

2.2.3 Alasan Pemisahan payment-service

Data permintaan kantong darah dari rumah sakit membutuhkan format penyimpanan yang lebih fleksibel. Penggunaan MongoDB pada request-service memungkinkan kita untuk mengubah atau menambah bentuk data form permintaan di masa depan dengan mudah. Selain itu, pemisahan ini menjaga agar tansaksi permohonan darah tidak membebani database utama.

2.2.4 Perbandingan dengan Monolitik

Aspek	Monolitik	Microservice (Sistem Ini)
Scaling	Scale seluruh aplikasi	Scale per service
Deployment	Deploy ulang seluruhnya	Deploy independen
Teknologi	Satu stack teknologi	Bebas per service
Database	Satu database bersama	Database terpisah
Fault isolation	Satu titik gagal	Terisolasi per service

BAB 3 IMPLEMENTASI

3.1 REST API + Database

3.1.1 Struktur Database request-service (MySQL)

Jika layanan lain menggunakan MySQL, request-service memanfaatkan MongoDB untuk keluwesan skema data permohonan. Basis data blood_request_db memiliki collection bloodrequest yang menyimpan dokumen dengan spesifikasi:

- Menyimpan informasi rumah sakit: hospital_name
- Detail darah: blood_group (A, B, AB, O), rhesus (+, -), dan volume_ml.
- Informasi pemohon: requested_by_email.
- Status permohonan: status (menunggu persetujuan/disetujui/ditolak) dengan nilai default pending.

3.1.2 Paging dan Filtering

Endpoint GET /api/stocks pada layanan donor mengimplementasikan sistem paging bawaan dari Laravel untuk membatasi jumlah data yang ditampilkan per halaman, sehingga performa API tetap ringan meskipun data stok darah berjumlah ribuan.

3.1.3 HTTP Method dan Response Code

Method	Endpoint	Response Sukses	Response Error
POST	/api/auth/register	201 Created	400, 500
POST	/api/auth/login	200 OK	401, 500
GET	/api/auth/oauth/google	200 OK / Redirect	400, 500
GET	/api/stocks	200 OK	500
POST	/api/requests	201 Created	401, 403, 500
GET	/api/requests	200 OK	401, 403, 500

3.2 REST API PHP MVC (Laravel 11)

Donor-service dibannngun menggunakan Laravel 11 dengan pola arsitektur MVC yang jelas:

- Model: Menggunakan Eloquent ORM untuk memetakan data stok kantong darah dari basis data relasional (MySQL).
- Controller: Berisi logika bisnis untuk menarik, menyaring, dan menyajikan data ketersediaan stok darah kepada publik melalui endpoint API.
- Routes: Didefinisikan pada routes/api.php agar secara spesifik menangani lalu lintas REST API.

3.3 Otentikasi JWT

auth-service mengimplementasikan autentikasi berbasis JWT dengan ketentuan:

- Access token di-generate menggunakan algoritma HS256 melalui library jsonwebtoken setelah pengguna berhasil memvalidasi email dan password.
- Middleware JWT diterapkan secara terpusat di API Gateway dan per-rute di request-service.
- Payload JWT menyimpan data sensitif milik user secara terenkripsi (seperti email dan ID Akun).

3.4 Otorisasi OAuth 2.0

Sistem mengimplementasikan Google OAuth 2.0 dengan Authorization Code Flow:

1. Client diarahkan (redirect) ke GET /api/auth/oauth/google.
2. User melakukan persetujuan (consent) di halaman resmi akun Google.
3. Google mengembalikan user ke callback URL dengan membawa authorization code.
4. Server menukar kode tersebut dengan access token ke API Google (oauth2.googleapis.com).
5. Server menarik data profil pengguna (nama, email) dan menghasilkan Token JWT internal sistem untuk diberikan kepada client.

3.5 API Gateway

API Gateway diimplementasikan menggunakan Node.js + Express sebagai single entry point dengan fitur:

- JWT Validation, memvalidasi token sebelum meneruskan permintaan ke service yang sensitif.
- Rate Limiting, membatasi maksimal 60 permintaan per menit per alamat IP menggunakan modul express-rate-limit.
- Proxy Routing, meneruskan permintaan ke service yang tepat menggunakan modul express-http-proxy.

3.5.1 Peta Routing Gateway

Path Pattern	Service Tujuan	Auth
/api/auth/register, /login	auth-service:3001	Public
/api/auth/oauth/google/**	auth-service:3001	Public
GET /api/stocks/**	donor-service:3002	Public
POST /api/request/**	request-service:3003	JWT Required
GET /api/requests/**	request-service:3003	JWT Required

3.6 Keamanan dan Manajemen Hak Akses

Alih-alih menggunakan hierarki role yang rumit, sistem donor darah ini menekankan pada pemisahan tingkat privasi endpoint:

- Akses Public: Endpoint yang ditujukan untuk transparansi informasi, seperti melihat ketersediaan stok darah (/api/stocks). Publik dapat mengaksesnya tanpa token.
- Akses Protected: Endpoint transaksional, seperti membuat permintaan darah dari rumah sakit (/api/requests). Akses ini sangat dibatasi dan diwajibkan menyertakan Token JWT berformat Bearer pada Authorization Header, sehingga sistem mengetahui secara pasti identitas (email) entitas yang melakukan request.

BAB 4 STRUKTUR REPOSITORY

4.1 Struktur Folder

Repository GitHub: https://github.com/pierevalkyrie/uts-pplos-b-2410511152

```
uts-pplos-b-2410511152/
│
├── gateway/
│   ├── src/
│   │   └── index.js        # Entry point API Gateway & Routing
│   ├── .env                # Konfigurasi port dan URL antar service
│   └── package.json
│
├── services/
│   ├── auth-service/       # Node.js + Express + MySQL
│   │   ├── src/
│   │   │   ├── controllers/ # Logika Register, Login, & OAuth
│   │   │   ├── routes/      # Endpoint /api/auth
│   │   │   └── index.js     # Entry point Auth Service
│   │   └── .env
│   │
│   ├── donor-service/      # Laravel 11 + MySQL
│   │   ├── app/Http/Controllers/ # Logika cek ketersediaan stok
│   │   ├── routes/
│   │   │   └── api.php      # Endpoint /api/stocks
│   │   └── .env
│   │
│   └── request-service/    # Node.js + Express + MongoDB
│       ├── src/
│       │   ├── config/      # Koneksi ke MongoDB (db.js)
│       │   ├── models/      # Skema BloodRequest
│       │   ├── routes/      # Endpoint /api/requests
│       │   └── index.js     # Entry point Request Service
│       └── .env
│
├── docs/
│   └── laporan-uts.pdf     # Laporan ini
├── README.md               # Dokumentasi dan Link Video Demo
└── .gitignore
```

4.2 Konvensi Commit

Repository menggunakan format Conventional Commits dengan minimal 15 commit yang tersebar dalam 7 hari:

- feat: penambahan fitur baru
- fix: perbaikan bug
- docs: perubahan dokumentasi
- refactor: refactoring kode tanpa perubahan fungsional

- chore: perubahan konfigurasi atau setup

Branch yang digunakan dalam pengembangan sistem ini meliputi: main, feature/auth, feature/oauth, feature/donor-service, feature/request-service, dan feature/gateway. Setiap branch fitur di-merge ke main melalui mekanisme Pull Request.

BAB 5 PENGUJIAN

5.1 Alur Pengujian

Pengujian fungsionalitas API dilakukan menggunakan Postman dengan collection yang telah disediakan i dalam folder /docs/postman/. Semua request diarahkan melalui API Gateway di port 8000. Alur pengujian lengkapnya adalah sebagai berikut:

1. Health Check Gateway: Memastikan API Gateway berjalan dan merespons dengan baik.
2. Register User Baru: Mengirim data user (nama, email, password) ke POST /api/auth/register.
3. Login User: Mengirim email dan password ke POST /api/auth/login untuk mendapatkan dan meyimpan access_token (JWT)
4. Cek Ketersediaan Stok Darah (Public): Melakukan GET /api/stocks untuk menarik data inventaris kantong darah dari Laravel (donor-service) tanpa menggunakan token.
5. Membuat Permintaan Darah (Protected): Melakukan POST /api/requests dengan melampirkan Token JWT di header Authorization, berisi data request seperti hospital_name, blood_group, dan volume_ml.
6. Melihat Riwayat Permintaan: Melakukan GET /api/requests (dengan token) untuk memastikan data permohonan darah berhasil tersimpan di MongoDB (request-service).
7. Test Hak Akses (Unauthorized): Mencoba melakukan POST /api/requests tanpa menyertakan Token JWT untuk memastikan middleware Gateway bekerja memblokir akses (Expected: 401 Unauthorized).
8. Test Google OAuth 2.0: Menguji endpoint otorisasi dengan membuka GET /api/auth/oauth/google melalui pelayar web (browser), memiliki akun Google, dan memastikan sistem memulangkan data profil beserta token pada halaman callback.

5.2 Screenshot

Bukti screenshot lengkap dari setiap proses pengujian endpoint di atas dilampirkan pada file README.md dan arsip gambarnya tersedia di dalam folder docs/screenshots/. Terdapat juga link video demonstrasi jalannya aplikasi yang disertakan pada bagian awal dokumentasi repositori.

BAB 6 KESIMPULAN

6.1 Kesimpulan

Sistem Donor Darah berbasis *microservice* ini berhasil dibangun dengan memenuhi seluruh poin persyaratan UTS:

- Arsitektur sistem terdiri dari 3 microservice independen (auth-service, donor-service, dan request-service) serta 1 API Gateway sebagai single entry point, di mana masing-masing layanan memiliki basis data terpisah (Database per Service).
- Implementasi REST API menggunakan basis data relasional MySQL pada auth-service dan donor-service, serta MongoDB pada request-service untuk menyimpan data permohonan darah yang fleksibel.
- donor-service berhasil dibangun menggunakan PHP MVC (Laravel 11) dengan pemisahan lapisan Model, Controller, dan Route yang jelas untuk mengelola stok darah.
- Autentikasi JWT diimplementasikan dengan fitur access token untuk mengamankan akses ke layanan permohonan darah, dilengkapi dengan mekanisme validasi di level Gateway.
- Otorisasi OAuth 2.0 menggunakan Google sebagai penyedia (provider) telah berhasil diterapkan dengan alur Authorization Code Flow, yang memungkinkan pengguna masuk menggunakan akun Google mereka.
- API Gateway berhasil mengimplementasikan fungsi routing antar layanan, validasi JWT secara terpusat, dan rate limiting untuk membatasi maksimal 60 permintaan per menit guna menjaga stabilitas sistem.

6.2 Saran Pengembangan

Untuk meningkatkan kualitas dan performa sistem di masa mendatang, berikut adalah beberapa saran pengembangan yang dapat dilakukan:

- Implementasi Docker dan Docker Compose untuk mempermudah proses penyebaran (deployment) dan manajemen kontainer di lingkungan produksi.
- Penambahan pengujian otomatis berupa unit test dan integration test pada setiap layanan untuk menjamin kualitas kode.
- Implementasi message queue (seperti RabbitMQ atau Kafka) untuk mendukung komunikasi asynchronous antar layanan, misalnya untuk sinkronisasi otomatis antara stok darah dan data permintaan.
- Penambahan fitur notifikasi melalui email atau layanan pesan lainnay ketika permintaan darah berhasil diproses atau disetujui.
- Implementasi sistem caching menggunakan Redis untuk mempercepat performa endpoint yang sering diakses, seperti pengecekan stok darah publik.